

Name : Sayan Das , Enrollment No: 12023052017057 , CSE(IOTCSBT) , Roll No: 57
Institute of Engineering & Management, Kolkata
Email ID: Sayan.das2023@iem.edu.in
me.sayandas03@gmail.com

Github Repository Link :

<https://github.com/SyntaxSayan/selenium-capstone>

Capstone Assignment 1: Automate a Web Application Using Selenium WebDriver with Python

Objective :

Design and implement a Selenium WebDriver automation solution for an E-Commerce application using Python and PyTest. The automation covers browser launch, user login, product search, product addition to cart, quantity update, cart verification, screenshot capture, external test-data handling, alert/popup handling, and HTML execution reporting.

Business Scenario

A customer wants to purchase a product from an E-Commerce site. The automation validates the complete shopping workflow from application launch through cart verification.

Application

TutorialsNinja E-Commerce Demo Application

<https://tutorialsninja.com/demo/>

Tools, Software, and Concepts Used

- Programming Language: Python
- Automation Tool: Selenium WebDriver
- Testing Framework: PyTest
- Design Pattern: Page Object Model (POM)
- Browsers: Google Chrome, Microsoft Edge, Firefox
- Test Data: Excel (.xlsx) and JSON
- Configuration: Configuration file (config.ini)
- Reporting: PyTest HTML Report (pytest-html)
- Utilities: Screenshot capture, alert/popup handling, Excel reader, JSON reader, custom logging
- Concepts: End-to-End Testing, Data-Driven Testing, Assertions, Waits, Exception Handling, Modular Automation

Source Code / Implementation

The automation project was implemented using a modular Page Object Model structure with separate configuration, page objects, test suites, utilities, test data, screenshots, logs, and reports.

AUTOMATION/

|

|— config/

```
| |— config.ini
| |— config_reader.py
|
|— test_data/
| |— test_data.json
| |— test_data.xlsx
| |— create_excel_data.py
|
|— pages/
| |— base_page.py
| |— home_page.py
| |— login_page.py
| |— search_results_page.py
| |— product_detail_page.py
| |— cart_page.py
|
|— utilities/
| |— alert_handler.py
| |— custom_logger.py
| |— excel_util.py
| |— json_util.py
| |— screenshot_util.py
|
|— tests/
| |— conftest.py
| |— test_e2e_purchase.py
| |— test_data_driven_excel.py
| |— test_data_driven_json.py
| |— test_cart_operations.py
|
|— reports/
```

```

|   └─ report.html
|
|   └─ screenshots/
|
|   └─ logs/
|
|   └─ automation.log
|
|   └─ pytest.ini
|
|   └─ requirements.txt
|
|   └─ run_tests.bat
|
|   └─ run_tests.ps1

```

Framework Components:

- BasePage provides reusable Selenium actions, waits, navigation, and common page-level functionality.
- HomePage handles application launch, product search, account navigation, and shopping-cart navigation.
- LoginPage handles authentication, login validation, warning messages, and automatic test-account registration when required.
- SearchResultsPage validates search results and adds the selected product to the cart.
- CartPage verifies cart contents, updates quantity, reads unit/total prices, dismisses alerts, and clears the cart.
- AlertHandler supports native JavaScript alerts as well as DOM/Bootstrap notification popups.
- ExcelUtil and JsonUtil load external test scenarios for data-driven execution.
- ScreenshotUtil stores timestamped screenshots for important test steps and failures.
- conftest.py manages WebDriver fixtures, browser/headless options, failure screenshots, and HTML-report metadata.

Source Code :

End-to-End Purchase Test (test_e2e_purchase.py)

```

@pytest.mark.e2e

class TestE2EPurchase:

    def test_complete_ecommerce_purchase_flow(self, driver):

        home_page = HomePage(driver)

        home_page.open()

        home_page.capture_screenshot("01_home_page_launched")


        login_page = LoginPage(driver)

        login_page.ensure_logged_in(

            email=ConfigReader.get_valid_email(),

            password=ConfigReader.get_valid_password(),

            firstname=ConfigReader.get_user_firstname(),

            lastname=ConfigReader.get_user_lastname(),

            telephone=ConfigReader.get_user_telephone()

        )

```

```

search_results_page = home_page.search_product("MacBook")
assert search_results_page.get_products_count() > 0
alert_msg = search_results_page.add_product_to_cart("MacBook")
assert "Success: You have added" in alert_msg

cart_page = CartPage(driver)
cart_page.open()
assert cart_page.is_product_in_cart("MacBook")

cart_page.update_quantity(new_qty=2, product_name="MacBook")
updated_qty = cart_page.get_quantity("MacBook")
unit_price = cart_page.get_unit_price("MacBook")
line_total = cart_page.get_total_price("MacBook")

assert updated_qty == 2
assert abs(line_total - round(unit_price * 2, 2)) < 0.05
cart_page.dismiss_alert()

```

Excel Data-Driven Test (test_data_driven_excel.py)

```

data = ExcelUtil.get_data_as_list_of_dicts(excel_path, "PurchaseScenarios")

@pytest.mark.parametrize("scenario", get_excel_test_data())
def test_ecommerce_excel_driven_scenario(self, driver, scenario):
    search_term = scenario.get("SearchTerm")
    product_name = scenario.get("ProductName")
    updated_qty = int(scenario.get("UpdatedQty", 2))

    home_page = HomePage(driver)
    home_page.open()
    search_page = home_page.search_product(search_term)
    assert search_page.get_products_count() > 0

    alert = search_page.add_product_to_cart(product_name=product_name)
    assert "Success: You have added" in alert

    cart_page = CartPage(driver)

```

```

    cart_page.open()

    cart_page.update_quantity(updated_qty, product_name)

    assert cart_page.get_quantity(product_name) == updated_qty

```

JSON Data-Driven Test (test_data_driven_json.py)

```

@pytest.mark.parametrize("case", get_json_test_data())
def test_ecommerce_json_driven_scenario(self, driver, case):
    search_term = case.get("search_term")
    product_name = case.get("product_name")
    updated_qty = int(case.get("updated_qty", 2))

    home_page = HomePage(driver)
    home_page.open()

    search_page = home_page.search_product(search_term)
    assert search_page.get_products_count() > 0

    alert = search_page.add_product_to_cart(product_name=product_name)
    assert "Success: You have added" in alert

    cart_page = CartPage(driver)
    cart_page.open()

    cart_page.update_quantity(updated_qty, product_name)

    assert cart_page.get_quantity(product_name) == updated_qty

```

Configuration and Dependencies

```

[common_info]

base_url = https://tutorialsninja.com/demo/

browser = chrome

headless = false

implicit_wait = 10

explicit_wait = 15

page_load_timeout = 30

[paths]

excel_data_path = test_data/test_data.xlsx

json_data_path = test_data/test_data.json

screenshot_dir = screenshots

report_dir = reports

log_file = logs/automation.log

```

```
requirements.txt  
selenium>=4.20.0  
pytest>=8.0.0  
pytest-html>=4.1.0  
pytest-metadata>=3.1.0  
openpyxl>=3.1.2  
webdriver-manager>=4.0.1
```

Test Data

The project uses two external test-data sources: an Excel workbook and a JSON file. Both contain purchase scenarios with product search terms, product names, initial quantities, updated quantities, unit prices, and expected totals.

- TC01 / TC_E2E_01: MacBook — quantity updated to 2 — expected total \$1,204.00.
- TC02 / TC_E2E_02: iPhone — quantity updated to 3 — expected total \$369.60.
- TC03 / TC_E2E_03: Samsung Galaxy Tab 10.1 — quantity updated to 2 — expected total \$483.98.

Test Scenarios Covered

1. Launch browser and open the TutorialsNinja application.
2. Authenticate a valid user; automatically create a fresh test account if the configured account is unavailable.
3. Search for products and verify matching product cards.
4. Add a selected product to the shopping cart and validate the success notification.
5. Open the cart and verify the selected product is present.
6. Update product quantity and validate the cart-update notification.
7. Verify quantity, unit price, line total, subtotal/final total, and mathematical consistency.
8. Handle and dismiss notification popups/alerts.
9. Verify invalid-login warning behavior.
10. Clear cart contents and verify the empty-cart message.
11. Execute equivalent purchase scenarios using Excel and JSON data.
12. Capture screenshots at important execution steps and on test failure.
13. Generate a self-contained HTML execution report.

Execution Report

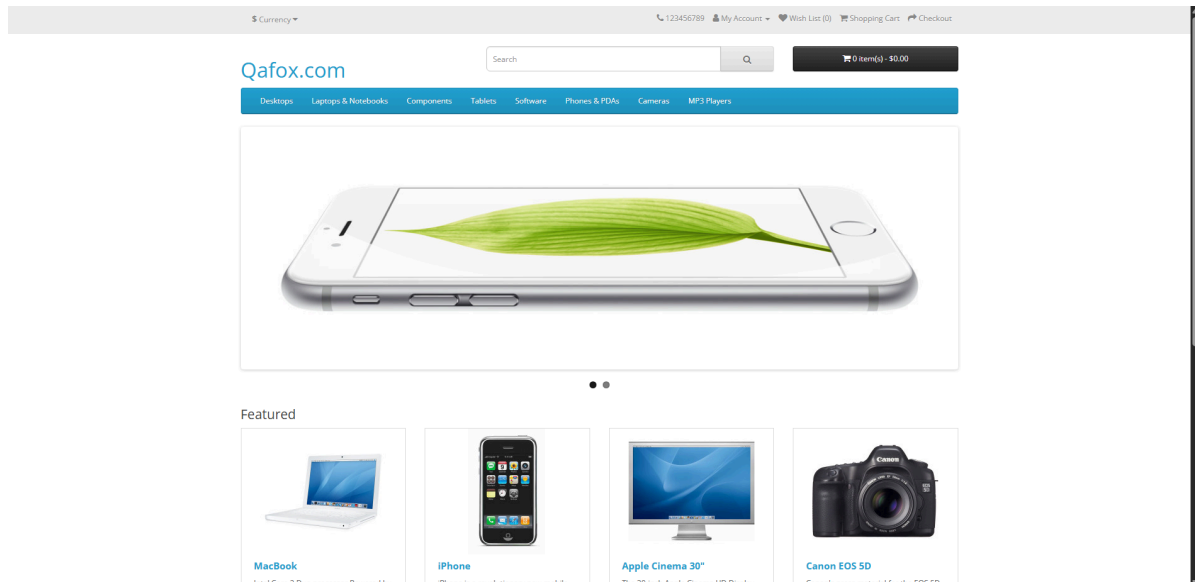
The uploaded project contains a generated PyTest HTML report at `reports/report.html`. The recorded execution summary in the project report shows:

- 10 tests passed.
- Total execution time: approximately 00:01:30.

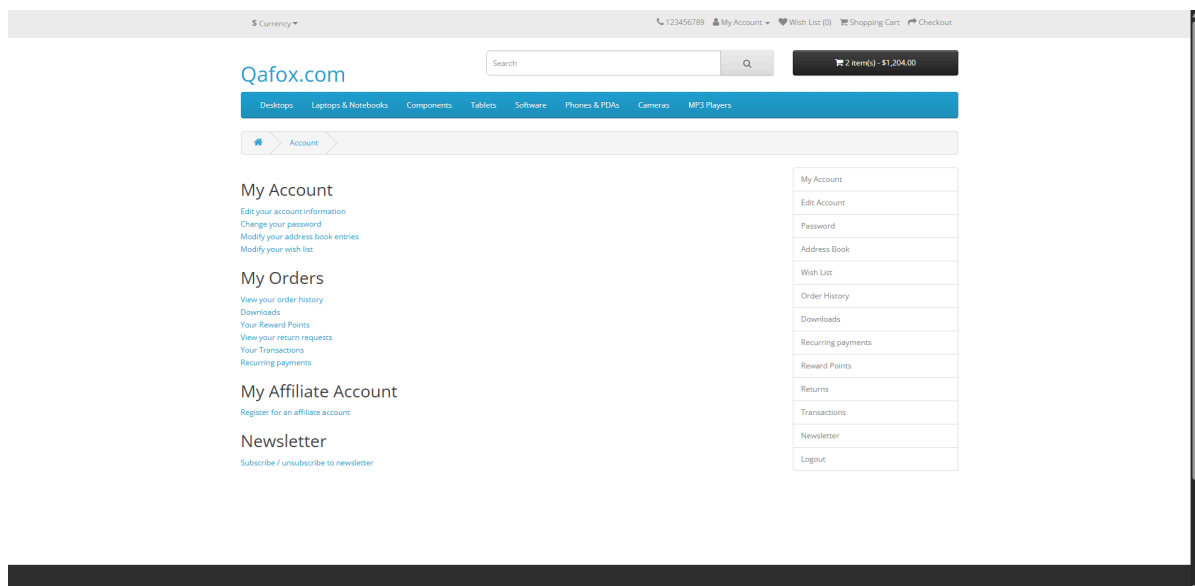
- The report includes environment metadata, test results, logs, and screenshot evidence.
- The execution covers E2E, Excel-driven, JSON-driven, cart-operation, and alert-handling scenarios.

Output with clearly labelled screenshots

1. Application Home Page Launched



2. User Logged In Successfully



3. Product Search Results

Currency

123456789My AccountWish List (0)Shopping CartCheckout

Qafox.com

MacBook

2 item(s) - \$1,204.00

DesktopsLaptops & NotebooksComponentsTabletsSoftwarePhones & PDAsCamerasMP3 Players

Search

Search - MacBook

Search Criteria

MacBook

All Categories

Search in subcategories

Search in product descriptions

Search

Products meeting the search criteria

Product Compare (0)

Sort By: Default

Show: 20



MacBook

Intel Core 2 Duo processor Powered by an Intel Core 2 Duo processor at speeds up to 2.16GHz, t...

\$602.00
Ex Tax:\$500.00



MacBook Air

MacBook Air is ultrathin, ultraportable, and ultra unlike anything else. But you don't lose in...

\$1,202.00
Ex Tax:\$1,000.00



MacBook Pro

Latest Intel mobile architecture Powered by the most advanced mobile processors from Intel, ...

\$2,000.00
Ex Tax:\$2,000.00

Search in product descriptions

Search

Products meeting the search criteria

Product Compare (0)

Sort By: Default

Show: 20



MacBook

Intel Core 2 Duo processor Powered by an Intel Core 2 Duo processor at speeds up to 2.16GHz, t...

\$602.00
Ex Tax:\$500.00

ADD TO CART



MacBook Air

MacBook Air is ultrathin, ultraportable, and ultra unlike anything else. But you don't lose in...

\$1,202.00
Ex Tax:\$1,000.00

ADD TO CART



MacBook Pro

Latest Intel mobile architecture Powered by the most advanced mobile processors from Intel, ...

\$2,000.00
Ex Tax:\$2,000.00

ADD TO CART

Showing 1 to 3 of 3 (1 Pages)

Information

About Us

Delivery Information

Privacy Policy

Terms & Conditions

Customer Service

Contact Us

Returns

Site Map

Extras

Brands

Gift Certificates

Affiliate

Specials

My Account

My Account

Order History

Wish List

Newsletter

Powered By OpenCart

5. Initial Shopping Cart View

Currency

123456789My AccountWish List (0)Shopping CartCheckout

Qafox.com

Search

3 Item(s) - \$1,806.00

DesktopsLaptops & NotebooksComponentsTabletsSoftwarePhones & PDAsCamerasMP3 Players

Shopping Cart

Products marked with *** are not available in the desired quantity or not in stock!

Shopping Cart (0.00kg)

Image	Product Name	Model	Quantity	Unit Price	Total
	MacBook *** Reward Points: 1800	Product 16	3 + -	\$602.00	\$1,806.00

What would you like to do next?

Choose if you have a discount code or reward points you want to use or would like to estimate your delivery cost.

Use Coupon Code

Use Gift Certificate

Sub-Total:	\$1,500.00
Eco Tax (2.00):	\$6.00
VAT (20%):	\$300.00
Total:	\$1,806.00

Continue ShoppingCheckout

6. Quantity Updated in Cart

Currency

123456789My AccountWish List (0)Shopping CartCheckout

Qafox.com

Search

2 Item(s) - \$1,204.00

DesktopsLaptops & NotebooksComponentsTabletsSoftwarePhones & PDAsCamerasMP3 Players

Shopping Cart

Success: You have modified your shopping cart!

Products marked with *** are not available in the desired quantity or not in stock!

Shopping Cart (0.00kg)

Image	Product Name	Model	Quantity	Unit Price	Total
	MacBook *** Reward Points: 1200	Product 16	2 + -	\$602.00	\$1,204.00

What would you like to do next?

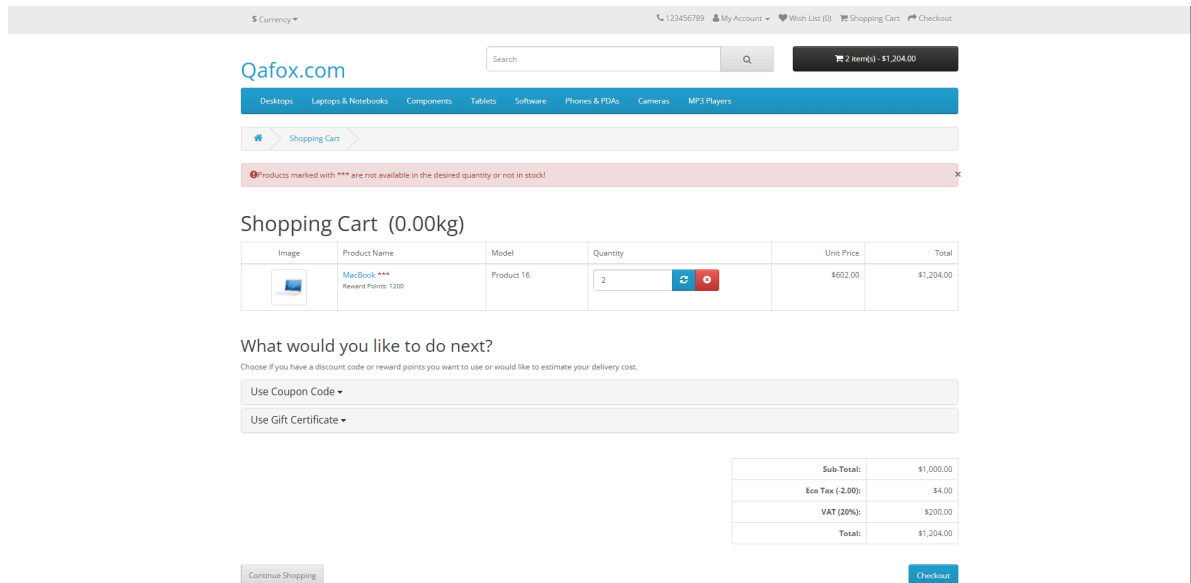
Choose if you have a discount code or reward points you want to use or would like to estimate your delivery cost.

Use Coupon Code

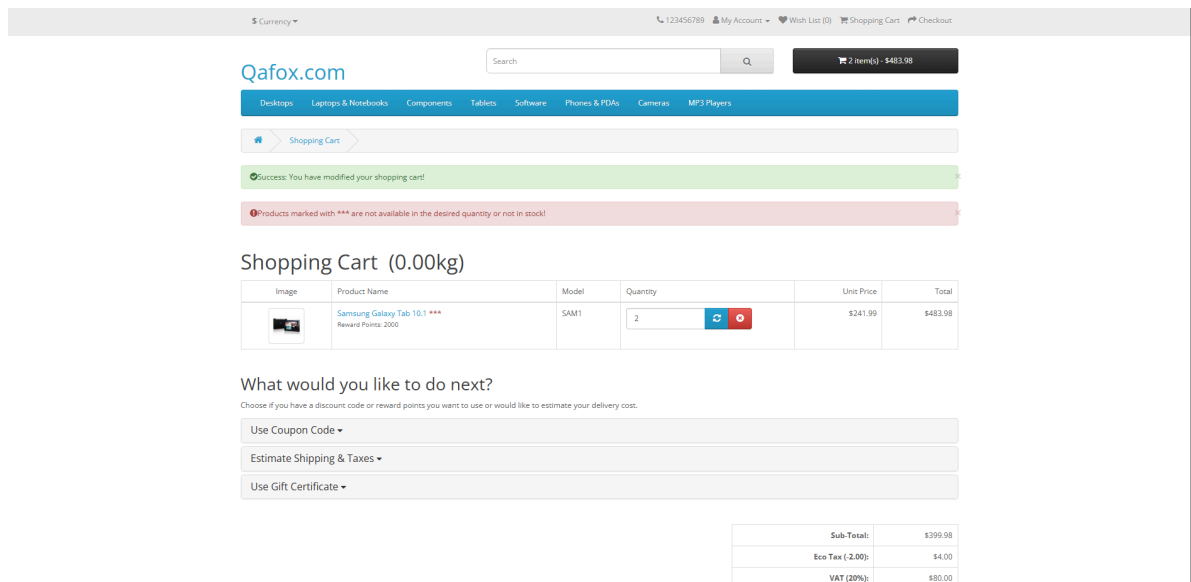
Use Gift Certificate

Sub-Total:	\$1,000.00
Eco Tax (2.00):	\$4.00
VAT (20%):	\$200.00
Total:	\$1,204.00

7. Cart Verified and Alert Dismissed



8. JSON Data-Driven Quantity Verification



Result

The Selenium Python automation project successfully automated the requested E-Commerce customer purchase workflow on the TutorialsNinja demo application.

The framework was able to:

- Launch the browser and open the application.
- Log in to the application and validate authentication.
- Search for products and verify search results.
- Add products to the shopping cart.
- Update product quantity and validate the update message.
- Verify cart quantity, unit price, line total, and final cart total.
- Read test scenarios from both Excel and JSON files.
- Handle notification alerts and popup messages.
- Capture clearly labelled screenshots and failure screenshots.
- Generate a self-contained HTML test execution report.

Observation

- The Page Object Model separates page locators and actions from test logic, making the automation easier to maintain.
- PyTest parameterization allows the same purchase workflow to execute against multiple external data rows.
- Excel and JSON test data reduce the need to hard-code product scenarios inside the test cases.
- Explicit waits and reusable BasePage methods improve synchronization with dynamic web elements.
- AlertHandler provides a reusable mechanism for JavaScript alerts and DOM/Bootstrap notification popups.
- Screenshots provide visual evidence of the important stages of execution and help diagnose failures.
- Configuration management centralizes the application URL, browser, timeouts, test data paths, screenshot directory, report directory, and logging path.
- The HTML report gives a consolidated view of the test execution results and environment information.

Conclusion

The project successfully demonstrates an end-to-end Selenium WebDriver automation solution using Python and PyTest for an E-Commerce application. It covers the complete customer purchase workflow required by Capstone Assignment 1 and extends the solution with Page Object Model architecture, Excel and JSON data-driven testing, reusable utility classes, alert handling, screenshot evidence, logging, configuration management, and HTML reporting. The recorded project execution contains 10 passed tests, demonstrating the implemented automation flow and supporting test scenarios.